SIC Lab
Asymmetric cryptography

version 1.0

Authors: André Zúquete, Vitor Cunha

Version log:

- 1.0: Initial version

1 Introduction

In this guide we will develop programs that use cryptographic methods, relying in the **crypto** library.

2 RSA

2.1 RSA key pair generation

An RSA key pair is formed by two pairs of values: the private key – (d, n) – and the public key – (e, n). The n is the modulus, d and e are the private and public exponents, respectively.

The values have some relation between themselves, which is related with the way they are generated. The generation depens on one dimension, which is the number of bits of n (or the index of its most significative bit). Given this value, we generate two random, prime values with a dimension close to n, p and q, and we compute $n = p \times q$. Then, we chose a suitable e and we compute d from p, q and e. Usually e is a Marsenne prime value (a prime number given by $2^x + 1$), such as 3 or $2^{16} + 1$ (65537).

In C, you can provide e and the dimension of n (key size) to generate an RSA key pair; p and q are randomly generated in order to be big, suitable to compute n with the appropriate dimension, and compatible with e:

```c
#include <openssl/evp.h>
#include <openssl/rsa.h>
#include <openssl/bn.h>

/*
 * out_key: this is the address (the pointer to) of a EVP_PKEY pointer
 * bits: the modulus length, in bits
 */

void
generate_rsa_key( EVP_PKEY ** out_key, int bits )
{
    EVP_PKEY_CTX * ctx;
    BIGNUM * e = NULL;
    OSSL_PARAM * params = NULL;
    int ok = 0;

    *out_key = NULL;

    // Create context to create an RSA key pair

    ctx = EVP_PKEY_CTX_new_id( EVP_PKEY_RSA, NULL );
    EVP_PKEY_keygen_init( ctx );

    // Create a Big Number with the desired value of e
```

```
    e = BN_new();
    BN_set_word( e, 65537UL );

    // Create a proper structure with the parameters to create an RSA key pair (e and modulus bits)

    params = set_rsa_params( e, bits );
    EVP_PKEY_CTX_set_params( ctx, params );

    // Create the key pair

    EVP_PKEY_keygen( ctx, out_key );

    // Free allocated structures

    BN_free( e );
    EVP_PKEY_CTX_free( ctx );
}
```

The code above does not implement error control (which is wrong!) but in this case there is no reason to get errors. So, we remove those verifications to make it more readable.

Also, the code above uses a a function to produce the RSA generation parameters as they are expected:

```
#include <openssl/evp.h>
#include <openssl/core_names.h>
#include <openssl/param_build.h>

OSSL_PARAM *
set_rsa_params( BIGNUM * e, int bits )
{
    OSSL_PARAM_BLD * param_bld = OSSL_PARAM_BLD_new();
    OSSL_PARAM * params;

    /*
     * RSA key generation parameters:
     *
     *   OSSL_PKEY_PARAM_BITS  -> modulus size in bits
     *   OSSL_PKEY_PARAM_RSA_E -> public exponent
     */

    OSSL_PARAM_BLD_push_uint( param_bld, OSSL_PKEY_PARAM_BITS, bits );
    OSSL_PARAM_BLD_push_BN( param_bld, OSSL_PKEY_PARAM_RSA_E, e );

    params = OSSL_PARAM_BLD_to_param( param_bld );

    // Free allocated structures

    OSSL_PARAM_BLD_free( param_bld );

    return params;
}
```

Again, the code above does not implement error control for the same reason as before.

Create a small program to generate an RSA key pair, with a key length specified by the user (1024, 2048, 3072 or 4096).

The program must save the key pair in two files, one for the private key and the other one for the public key, whose names should also be specified by the user. The keys can be saved as binary blobs but, probably, the PEM format will be more appropriate.

```
#include <stdio.h>
#include <openssl/evp.h>
#include <openssl/pem.h>

void
write_PEM_private_key( FILE * file_pointer, EVP_PKEY * pkey )
{
    /*
     * This writes an unencrypted private key in PEM format.
     *
     * To encrypt it, replace NULL with a cipher such as EVP_aes_256_cbc()
     * and provide a passphrase.
     */
```

```
    PEM_write_PrivateKey( file_pointer, pkey, NULL, NULL, 0, NULL, NULL );
}

void
write_PEM_public_key( FILE * file_pointer, EVP_PKEY *pkey )
{
    /*
     * PEM_write_PUBKEY writes the public key in SubjectPublicKeyInfo format:
     *
     *     -----BEGIN PUBLIC KEY-----
     *
     * This is usually preferred over the older RSA-specific public key format.
     */
    PEM_write_PUBKEY( file_pointer, pkey );
}
```

Again, no error control for the sake or legibility;

Run the program, several times, varying the key length and register the elapsed time using the `time` program in the shell (use `man time` for more information), for example:

```
time key_gen_rsa 4096 priv.pem pub.pem
```

Compare the size of the keys stored in each file (they are written in Base64). Notice that the private key is nearly 4 times bigger than the public one. This is caused by the fact that the private key contains not only d and n, but also p and q for accelerating private-key computations. And all these 4 values have a dimension similar to n. On the other hand, the public key file contains only e and n, and e is much smaller that n.

Questions:

- What do you think of using 4096 bit keys by default in relation to speed?
- How the actual key size varies with the number of bits?

2.2 RSA Encryption

Create a small program to encrypt a file using the RSA algorithm.

The user must indicate the following data:

1. Name of the original file to encrypt
2. Name of the file with the public key
3. Name for the encrypted file.

For the encryption select the OAEP padding scheme. Also, *pay attention to the size of the original file*, i.e., the file to encrypt. Using the OAEP default configuration, the block size is equal to the key size minus 42 bytes (for padding with SHA-1, calculated has $2 + 2. \times h$, where h is the size in bytes of the output of digest function used). So, for example using a 1024 bits RSA key (128 bytes), the block size is 86 bytes ($128 - 42$).

2.3 RSA Decryption

Create a small program to decrypt the contents of a file, whose name is provided by the user, using the RSA algorithm. The user must also indicate:

1. Name of the encrypted file;
2. Name of the file to save the decrypted content;
3. Name of the file containing the private key to use.

2.4 Large file encryption

As should be known by now, RSA encryption is not efficient and is not recommended to encrypt data bigger than its block size. Let's conduct a benchmark.

2.4.1 Symmetric vs. Asymmetric cryptography benchmark

Generate two files with 100 kB and 10 MB of size. Using these files and the `time` application, register the time it takes to encrypt and decrypt these files with RSA (key = 1024 bits) and AES-128. Employ the applications developed in this guide and in the previous one.

The following `bash` command can be used to generate a 100 kB file `file.txt`:

```
dd if=/dev/zero of=file.txt bs=1024 count=100
```

2.4.2 Symmetric and Asymmetric cryptography combination

Consider a large file (500 MB, for example) that you want to send to some person, with the guarantee that only that person can decrypt the file. More, you have the public key of that person, and you are not able to contact the person before sending the file.

So, you have a big file to transmit to a person, from which you know the public key, but the process will be very slow if you encrypt the file using RSA. However, you can use any other encryption algorithm to encrypt the file, but remember you want that only the receiver must be able to decrypt the file.

Questions: What combination of encryption technologies allow to efficiently send the file to the recipient, with the guarantee that only that person can decrypt the file?

A typical solution is called *Hybrid Encryption*, which combines two ciphers, a symmetric to encrypt the file with a random key, and a asymmetric to encrypt the key used in the previous step.

Implement a program that is able to encrypt/decrypt a large file using the AES-128 algorithm, employing a randomly generated secret. The secret should be supplied to the intended receiver ciphered with RSA using its own public key. The receiver should be able to reverse the process and obtain the original version of the large file. The program should accept the following arguments:

1. Operation to perform (encrypt or decrypt);
2. Name of the file with the key to be used (public or private);
3. Name of the (large) file to encrypt/decrypt (optional, you can use stdin).
4. Name of the output file (optional, you can use stdout).

Questions:

- With this method, what is sent to the destination?
- Should we always send the pubic key?

3 Elliptic curves

Elliptic curve cryptography (ECC) is a lot different from RSA.

First, whilst in RSA one chooses the length of the RSA modulus (in bits), in ECC one chooses a curve, and the curve defines the key length.

Second, the private and public keys that belong to the same key pair are of a different nature: the *private key is an integer* (a scalar), whilst the *public key is a two-coordinate point*. The public component is a point given by the multiplication of the private key with a so-called curve generator (G), which is a point of the curve. Since the sum of two points defines a new point (elliptic curves have a special algebra...), and the multiplication of G by the private key (a scalar) is nothing more than a sum of G values, it is simple to conclude that the public key is a point.

Third, there is no such thing as an encryption with a public key and a posterior decryption with the corresponding private key. This is not used at all. Instead, a method similar to a Diffie-Hellman key agreement is used. For encrypting a message to a receiver B, the sender A uses B's public and a new private key to compute a point in the curve, and from that point derives a secret, symmetric key. The message is

then encrypted with that symmetric key and sent to B, together with the public component of the random private key created by A. The receiver B multiplies the received public key with its own private key and reaches the same point in the curve, which it can use to derive the exact same secret, symmetric key and use it to decipher the message.

There are many elliptic curves, both proposed in standards and other publications. NIST proposed 15 curves, known as P, B and K curve (5 of each). The name stems from the mathematical elements used in the curves – P from prime, B from binary, K from Koblitz, a variant a B curves. Other curves are recommended by *de facto* standards, such as RFC 7748, which describes Curve25519 and Curve448). The SafeCurves publication provides some guidelines for selecting a god curve.

3.1 ECC key pair generation

ECC key pair generation is fairly simple. First, one chooses a curve; we will use P-521. Then, we generate a random private key for that curve. This is a 521-bit random integer. Then, from that private key, we generate the public key.

Create a program the generates a key pair and saves it in a file. The private component should be protected by a password, provided as a program parameter. You can use the PEM format for encapsulating each of the keys. We will assume this program will be called 'p521_gen'.

```
p521_gen password Kpair_priv Kpair_pub
```

Note: generate two PEM contents, one for the private component, another for the public component.

3.2 Secure messaging with public encryption and private decryption

Create a program 'ecc_encrypt' that encrypts the contents received from the standard input to the standard output. The program should have a parameter a file containing the public of the receiver.

The contents produced must contain both the encrypted input and the originator's public key. You can use any message format for that, for instance, JSON (do not forget to use base64 for encoding binary contents into strings for JSON). For symmetric encryption, you can use AES-128 in CTR mode and an IV derived from the secret key (e.g., by hashing it).

```
ecc_encrypt Kpub < plaintext > ciphertext
```

Create a program 'ecc_decrypt' that decrypts the contents received from the standard input to the standard output. The program should have as parameters a file containing the private key of the receiver and a password for having access to that private key. The format of the input is the one produced by 'ecc_encrypt'.

```
ecc_decrypt Kpriv password < ciphertext > recovered_plaintex
```

After these two command, the contents of 'plaintext' should be equal to the contents of 'recovered_plaintext'.

4 Further Reading

1. SafeCurves